

XBF-1 FLIGHT SIMULATOR

by Jeremy Mumia P15/142777/2021



Engine	Game Maker 7, GML(Game Maker Language)
--------	--

Type	3D Simulation, Flight Simulation,
------	-----------------------------------

Overview

A simple flight simulator game for a hypothetical XBF-1 aircraft. I have emulated basic flight functionality, take-off and landing systems, aerodynamics physics (drag and stall), HUD with meters and a blip map, fuel mechanic. Made over the course of 3 months.

Approach

1. Rudimentary approach to 3D by using D3D ray tracing for projecting 2D rooms into 3D worlds (3D illusion; **faux 3D**)
2. Use of simple low polygon models that are self made (like constraints of old consoles)
3. Generated my own sounds by programming **MIDI**

Challenges

1. 3D model generation via polygon geometry calculations instead of modelling software.
2. Understanding domain specific info on aircraft flight control
3. Using an older game engine that emulates some of the restraints that early innovators had to work under (esp in **3D**).

Deliverables

1. Game Executable
2. Video Demonstration
3. Documentation (you are here)



XBF-1 FLIGHT SIMULATOR ↙

Gist...

Aircraft basically have aerodynamics that generate lift. The amount of lift varies depending on the speed. A Cessna-172 aircraft (whose aerodynamic scheme I used as a guideline) basically has enough lift to **take off at 60kts**, going below this speed results in a **stall**. While landing, the basic idea is to **align to the runway, lose altitude** gradually and **shave off the speed** while maintaining the it above the **stall speed**. **Landing at speeds above 100kts is generally bad** for the airframe and can also result in the pilot overshooting the runway.

At their own discretion, the **pilot can trade off altitude to gain speed, or speed to gain altitude**.

Aerodynamic drag and energy conservation principles dictate that there will always be entropy of speed and altitude during flight.

A pseudo mechanism for fuel is implemented so the user of the simulator would need to land and stop the plane so as to refuel automatically.

There is flying Vrill UFO just to add interactivity and showcase complex modelling

XBF-1 FLIGHT SIMULATOR ↙

3D Code.

This is the code that changes the 2D projection to a 3D projection and sets up lighting, fog, draw distance, sky box and other configuration...

Execute Code

✓       Applies To: ☒ Self ☐ Other ☐ Object

```
1 draw_set_color(c_white);
2 d3d_set_fog(true, c_white, 200, room_width/5);
3 draw_set_alpha(1);
4 d3d_draw_floor(0, 0, -0.1, room_width, room_height, -0.1, background_get_texture(land), 5, 5)
5
6 d3d_set_lighting(true);
7 d3d_set_shading(true);
8 d3d_light_define_direction(0, 1, 1, -1, c_white);
9 d3d_light_enable(0, true);
10 d3d_light_define_direction(3, -0.5, -0.5, 0, c_white);
11 d3d_light_enable(3, true);
12 d3d_light_define_ambient(c_dkgray);
13
14 d3d_light_enable(1, true);
15
16 d3d_set_culling(false); //
17
18
19 d3d_draw_block(room_width/2, room_height/2, 0, (room_width/2)+50, (room_height/2)+50, 500, background_get_texture(wtc), 0.5, 8);
20 d3d_draw_block(room_width/2-100, room_height/2-100, 0, (room_width/2)-50, (room_height/2)-50, 500, background_get_texture(wtc), 0.5, 8);
21
22
23 d3d_transform_set_identity();
24 d3d_transform_set_rotation_z(sky_angle);
25 d3d_transform_add_translation(room_width/2, room_height/2, 0);
26 d3d_draw_ellipsoid(-sky_radius, -sky_radius, -sky_radius, +(sky_radius), +(sky_radius), (sky_radius), background_get_texture(space), 1, 1, 24);
27 d3d_transform_set_identity();
```

XBF-1 FLIGHT SIMULATOR ↙

Plane Code.

I implemented various scripts for the functionality of the plane. They are attached as ship-*.txt files together with this documentation.

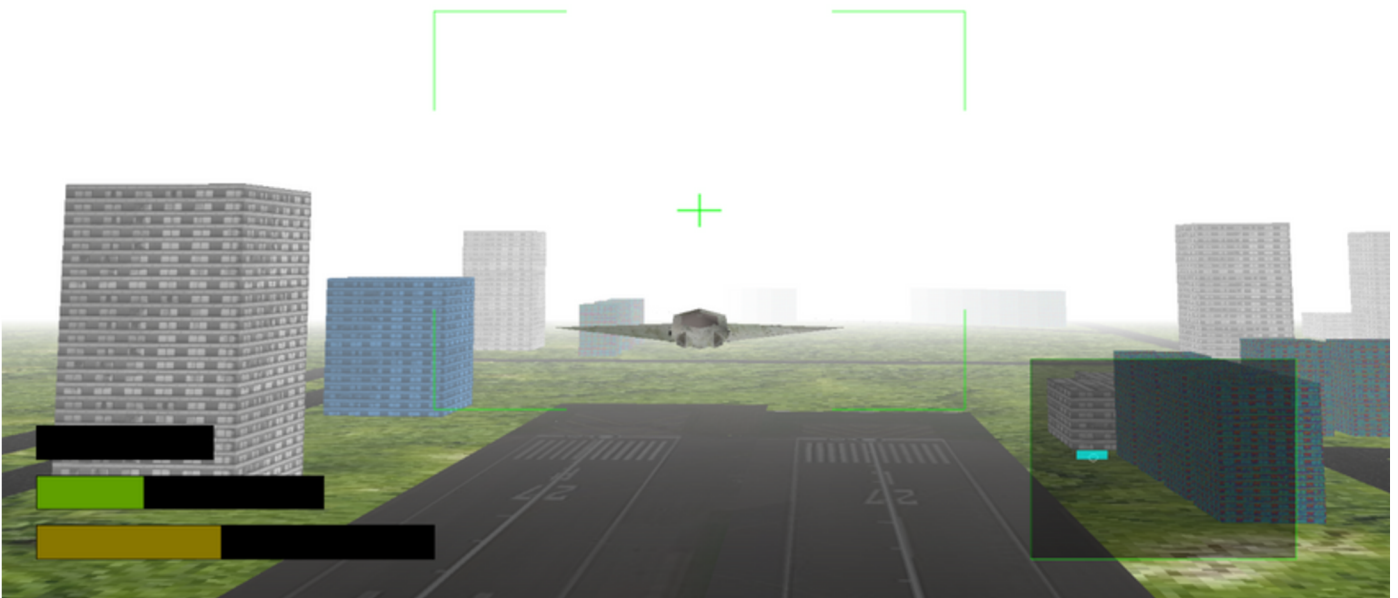
There is code for the 3D model of the ship, code for the controls and code for collision events. These are primarily in GML, a C-like or JS-like syntax language for use with Game Maker 7.

XBF-1 in action below...

Speed: 90.02 kts

Altitude: 533.50 ft

Calibration: QUICK

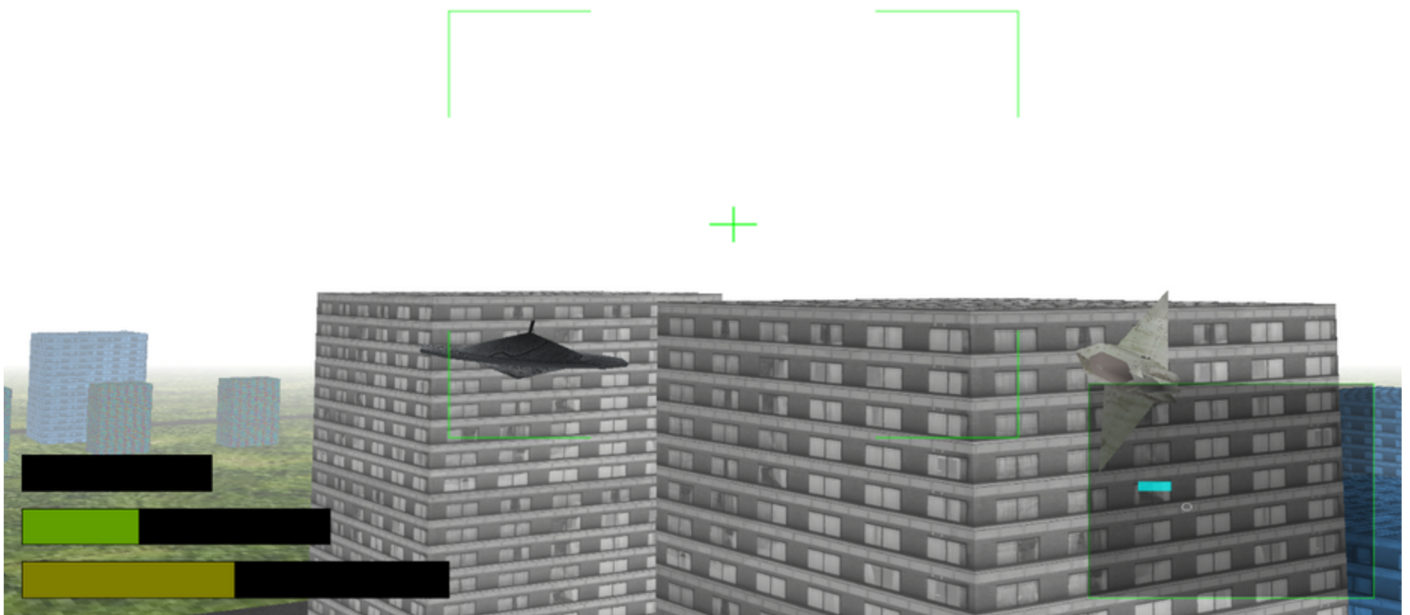


XBF-1 FLIGHT SIMULATOR ↙

Vril UFO Model.

The UFO model code is attached as UFO-model.txt file together with this documentation.

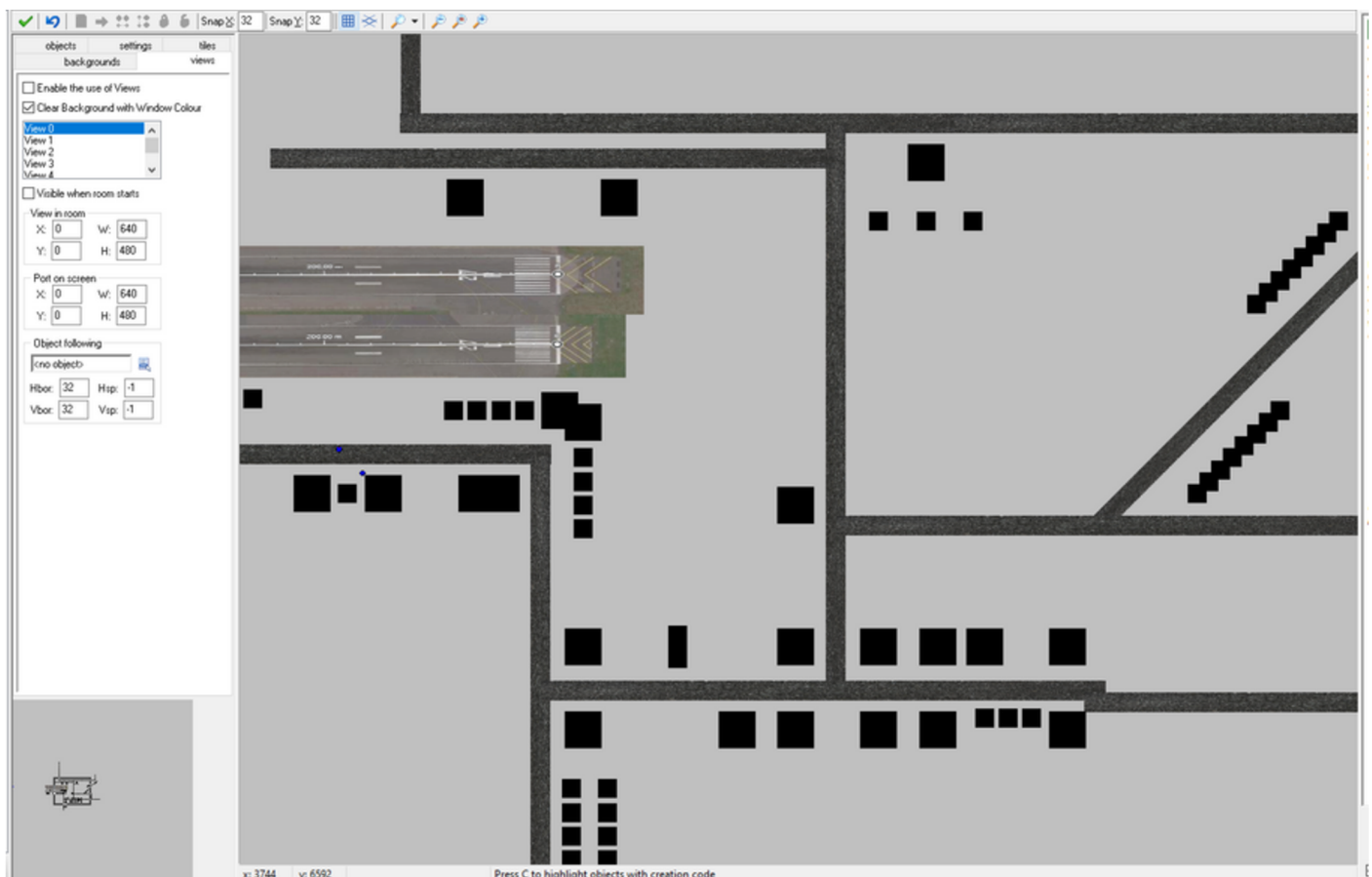
It is a composite of various ellipsoid and polygon shapes to come up with its design. Below is a Vril UFO spotted during flight...



XBF-1 FLIGHT SIMULATOR ↙

Level Design

Game Maker 7 comes with a room editor which allows placement of game objects and tilesets which make up the graphic resources and some controllers.



XBF-1 FLIGHT SIMULATOR ↙

Sound Design

I used a JavaScript Library called midi-writer-js to generate the plane engine sounds in the game as per the snippet below...

```
JS plane.js
1  const MidiWriter = require('midi-writer-js');
2
3  // Create a new track
4  let track = new MidiWriter.Track();
5
6  // Set a slower tempo for the engine hum, you can adjust as needed
7  track.setTempo(90); // Lower tempo to represent slower engine hum
8
9  // Add a program change event to select an instrument
10 // Use a synthesizer pad or bass sound for the low-end engine rumble
11 track.addEvent(new MidiWriter.ProgramChangeEvent({instrument: 89})); // 89 = Pad 2 (Warm), or try 81 = Lead 1 (Square)
12
13 // Create the Low-pitched engine rumble
14 const engineRumble = new MidiWriter.NoteEvent({
15   pitch: ['C2'], // Very Low note for deep engine hum
16   duration: '1', // Long sustained note for a continuous sound
17   velocity: 70 // Medium intensity for a deep, resonant sound
18 });
19
20 // Add a high-pitched futuristic oscillation layer
21 const highOscillation = new MidiWriter.NoteEvent({
22   pitch: ['G5', 'A5'], // Higher tones to give a techy feel
23   duration: '8', // Shorter duration to create oscillation effect
24   velocity: 50 // Softer sound, creating a contrast with the rumble
25 });
26
27 // Pitch bend event to simulate an evolving, dynamic engine sound
28 track.addEvent(new MidiWriter.PitchBendEvent({value: 8191, tick: 64})); // Pitch up halfway through
29
30 // Combine events for a futuristic sound
31 for (let i = 0; i < 16; i++) {
32   track.addEvent(engineRumble); // Repeated engine hum
33   track.addEvent(highOscillation); // Repeated high-pitched oscillations
34 }
35
36 // Write to a MIDI file
37 let write = new MidiWriter.Writer(track);
38 require('fs').writeFileSync('futuristic-plane-engine-midi.mid', write.buildFile());
39 console.log('MIDI file generated: futuristic-plane-engine-midi.mid');
40
```

XBF-1 FLIGHT SIMULATOR ↙

THATS ALL! (For the Demo Version)

Here is how the game splash screen looks...

